

超算概述：HPC 入门讲义

目 录

1 引言：为什么我们需要超算？	2
1.1 云计算：算力如自来水	2
1.2 超算的应用领域	2
2 超算竞赛与浙大超算队	2
2.1 世界三大超算竞赛	2
2.2 HPL 与 Top500 榜单	3
2.3 浙大超算队的光辉历程	3
2.4 经典案例：SKA Gridding 优化	4
3 并行计算：超算的核心思想	4
4 计算机基础回顾	5
4.1 数据表示：二进制、原码与补码	5
4.1.1 原码：最直观的表达	5
4.1.2 补码：让减法变成加法	5
4.1.3 原码乘法的易错点	6
4.2 存储器层次结构	6
4.2.1 存储单位换算	7
4.3 计算机系统分层与指令周期	7
5 网络基础	7
5.1 TCP/IP 协议	8
5.2 IP 地址与子网掩码	8
5.2.1 IP 地址	8
5.2.2 专用地址范围	8
5.2.3 子网与子网掩码	9
5.3 CIDR：无类别域间路由	9
6 操作系统核心功能	9
7 要点速查	10
8 本章你将学会	10
9 小结	11

1 引言：为什么我们需要超算？

你用过手机上的天气预报吗？短短几秒就能告诉你明天会不会下雨。但在这背后，气象中心需要处理全球数十年的气温、湿度、风速数据，计算量之大远超你手机的承受能力。当单个计算任务的规模和复杂度超出单机能力时，我们就需要**超级计算**（High Performance Computing, HPC）的力量了。

超级计算，简称超算，是利用大量处理器（CPU/GPU）协同完成大规模科学计算与数据处理的计算方式。与日常使用的个人电脑不同，超算集群由成百上千个计算节点组成，每个节点又包含多个多核处理器。超算追求的核心目标可以概括为两个词：“算得快”和“算得多”。

1.1 云计算：算力如自来水

直觉 不妨把超算想象成一座“自来水厂”。你拧开水龙头，水就来了，你不需要知道水从哪条河来、经过哪个水厂净化。同样，云计算平台把计算资源像自来水一样按需供应：你打开一台终端，就能使用虚拟机的算力，支付费用即可，但不需要知道计算资源来自哪里。云平台的底层，依赖的正是超算级别的计算、存储与网络能力。

超算不仅是科研利器，更已成为与电、自来水、通信、燃气并列的“第五大基础设施”。在数字经济时代，**算力**（computing power）代表着“算得快”的能力，**网络**（network）代表着“跑得快”的能力，两者共同支撑着人工智能、大数据、物联网等新一代信息技术。

1.2 超算的应用领域

超级计算机广泛应用于以下领域：

- 1. **地球气候模拟**：处理全球数十年的气象数据，预测气候变化
- 2. **宇宙天体研究**：模拟星系演化、黑洞碰撞等天文现象
- 3. **生物医药研制**：筛选亿万级分子组合，加速新药研发
- 4. **新材料开发**：在原子层面模拟材料属性
- 5. **电网可靠性模拟**：保障大规模电力系统的稳定运行
- 6. **人工智能**：大模型训练需要万亿级参数的梯度更新，直接推动了 AI 的新一轮发展高潮

正因为超算在科研和产业中的巨大价值，超级计算机如今已成为衡量一个国家科技与经济实力的重要指标。中国在硬件上已进入世界超算大国之列，但在应用软件和人才储备方面仍在持续投入。

2 超算竞赛与浙大超算队

学习超算不仅仅是看书听课，**以赛代练**是最有效的成长路径。全球有三大顶级大学生超算竞赛，它们是超算领域最高水平的竞技舞台。

2.1 世界三大超算竞赛

三大竞赛各有特色，但共同点是：难度极高，获奖率低于 5%。

竞赛	发起地	起始年	特点
ASC	中国	2012	全球规模最大，预赛 200+ 队伍，决赛 20 队
ISC	德国	2012	与 HPCAC 合作，约 10+ 队
SC/SCC	美国	2007	顶队云集，约 10+ 队

ASC（世界大学生超级计算机竞赛，ASC Student Supercomputer Challenge）肇始于2012年，依托于亚洲超算协会，在中国教育部和科技部的大力支持和指导下举办，迄今影响力不断攀升，目前已成为全球规模最大、影响范围最广的超算赛事。2017年ASC共有来自五大洲十八个国家的230支参赛队报名，参加人数超过1100人；2018年更扩展到25个国家300多支队伍。

竞赛采用预赛加决赛双赛制，评审专家由中国科学院、各国超算中心负责人经逐轮筛选、会议讨论和投票表决完成评审。奖项设置包括：

1. 总冠军（100K RMB）
2. 亚军（50K RMB）
3. **e Prize 计算挑战奖**（27.1828K RMB），指定应用进行大规模并行优化
4. 最高计算性能奖（10K RMB）
5. 应用创新奖（10K RMB）

其中 e Prize 被誉为“青年人才的戈登·贝尔奖”（Gordon Bell Prize），鼓励青年人才大胆采用创新技术，勇于解决超算应用难题，敢于挑战计算性能极限。

2.2 HPL 与 Top500 榜单

要衡量一台超级计算机有多快，业界使用 HPL（High-Performance Linpack）作为标准基准测试。HPL 用于求解稠密线性方程组，衡量超级计算机的浮点峰值性能。全球最快的 500 台超级计算机排行榜称为 **Top500**，每年发布两次，HPL 性能是排名的核心指标。

Top500 榜单可在 <http://www.chinastor.com/hpc-top500/> 查询。例如 2022 年 11 月榜单中，Top1 为 Summit，拥有约 239 万余个核心。

2.3 浙大超算队的光辉历程

浙江大学超算队（ZJUSCT）成立于 2013 年 10 月，源于浙江大学计算机科学与技术学院下属的高性能计算兴趣小组。超算场地在紫金港校区东四教学楼 5 楼，目前有常任指导老师 4 名、队员 20 余名。

自 2014 年首次参加 ASC 以来，浙大超算队累计获得 27 项奖项，其中最耀眼的是多次获得“世界最高计算性能冠军奖”并打破世界纪录：

年份	代表性成绩
2014	ASC 一等奖（中山大学）
2015	ASC 一等奖（太原理工大学）
2016	ASC 一等奖 + HPL 冠军，破世界纪录（华中科技大学）
2018	ASC 一等奖（南昌大学）
2020-21	ASC 一等奖（南方科技大学）
2023	ASC 一等奖 + HPL 冠军，破世界纪录（中科大）
2024	ASC 一等奖 + 最高计算性能冠军（上海大学）
2025	ASC 一等奖 + 应用创新奖（青海大学）

此外，浙大超算队在 ISC（欧洲赛）和 SC（美国赛）中也屡获佳绩：SCC22 单项第一总排名第四，SCC23（Indy）全球总冠军，ISC24 全球第五。

浙大超算队的成果不止于竞赛。例如 SCC22 的比赛成果形成论文发表在国际顶级期刊 *IEEE TPDS 2023* (CCF-A 类)。团队还与中科院合作开展格点粒子物理学计算优化，联合发表多篇论文。

2.4 经典案例：SKA Gridding 优化

例 在 ASC15 竞赛中，e Prize 题目是 SKA（平方公里射电望远镜阵列，Square Kilometre Array）的核心应用 Gridding。Gridding 是 SKA 中计算步骤最复杂、耗时最多的数据处理环节。

中山大学团队在 16 个基于 CPU+MIC 节点的超算平台上，对 Gridding 程序进行并行化及性能优化。他们重点优化了内存访问效率，以及 CPU 和 MIC 的并行计算效能。

最终结果：实现了 **600 倍加速**，完成同等规模数据处理的系统功耗降至原来的 1/20。这就是“与科研亲密接触，挑战世界难题”的真实写照，也展示了 HPC 优化的巨大潜力。

3 并行计算：超算的核心思想

超算之所以“算得快”，核心秘密在于**并行计算**（Parallel Computing）：将一个计算任务拆分到多个处理单元上同时执行，从而缩短总运行时间。这是贯穿整个 HPC 学习的核心概念，后续每一章都会围绕它展开。

直觉 不妨想这样一个场景：食堂要做 9 道菜。如果只有 1 个大厨，他得一道一道做，总共花很长时间。但如果你请了 9 个大厨，每人做一道菜，各做各的，互不干扰，那总时间就大大缩短了。这就是并行计算的直觉：把大任务拆成小任务，分头同时干。

但并非所有任务都能这么简单地并行。我们来看两个例子：

例 1：矩阵逐元素加法（embarrassingly parallel）

有一个 $N \times N$ 的矩阵，要对每个元素做加法 $C_{i,j} = A_{i,j} + B_{i,j}$ 。每个位置的计算互相独立，因此可以“9 个核心各处理一部分位置”，完美并行。这就像 9 个大厨各做一道菜，无需配合。

例 2：向量点积（需要规约）

计算两个长度为 N 的向量的点积 $\text{dot}(a, b) = \sum_{i=1}^N a_i b_i$ 。乘法 $a_i b_i$ 可以并行（分头算乘积），但最后的累加需要串行规约，大厨之间需要配合：每人算完自己的乘积后，还要汇总求和。

这里引出了一个重要的分类：**embarrassingly parallel**（令人尴尬的并行）指任务间几乎没有依赖，天然适合并行；而需要规约、通信的任务则需要更复杂的并行策略。

HPC 的三大并行技术栈贯穿本课程，它们解决不同层次的并行问题：

1. **OpenMP**（开放式多处理）：多线程并行，解决“一台机器内多个核怎么协同”
2. **MPI**（消息传递接口，Message Passing Interface）：多进程通信，解决“多台机器怎么协同”

3. **CUDA**（统一计算设备架构，Compute Unified Device Architecture）：GPU 异构加速，利用 GPU 的大规模并行能力

后续章节将逐一深入这三大技术栈。现在你只需要记住：并行是超算的灵魂，三大栈是并行的工具。

4 计算机基础回顾

超算的基础是计算机系统。在深入并行编程之前，我们需要回顾几个关键概念：数据如何表示、存储如何分层、CPU 如何执行指令。这些看似基础的知识，在后续的性能优化中会反复出现。

4.1 数据表示：二进制、原码与补码

计算机内部一切信息都以 0/1 二进制位表示。一个数在计算机中的表示方式直接影响运算的正确性和效率。

4.1.1 原码：最直观的表达

原码（Sign-Magnitude）的规则很简单：最高位是符号位（0 表示正，1 表示负），其余位是数值的绝对值。

例 用 5 位原码表示 +11 和 -10：

0
+11 的原码：1011（符号位 0，数值位 1011 = 11）
1
-10 的原码：1010（符号位 1，数值位 1010 = 10）

原码虽然直观，但有一个致命问题：加减法需要先判断符号，再决定是做加法还是减法，运算电路复杂。于是**补码**（Two's Complement）应运而生。

4.1.2 补码：让减法变成加法

直觉 补码的核心思想是：让负数“变成”一个大的正数，使得加法电路可以直接处理减法。想象一个 12 小时制的时钟：如果现在 10 点，你想调到 7 点，可以往回拨 3 小时（减 3），也可以往前拨 9 小时（加 9）。在模 12 的世界里，-3 和 +9 是等价的。补码就是利用这个原理：在 n 位系统中， $-x$ 的补码就是 $2^n - x$ 。

正数的补码与原码相同；负数的补码是“原码符号位不变，其余位取反，然后加 1”。

例 用补码计算 $(+11) + (-10) = +1$

5 位系统中：

0
+11 的补码：1011（正数补码 = 原码）
1 1 1
-10 的补码：原码 1010 → 取反 0101 → 加 1 得 0110
加法运算（符号位参与运算，进位自然丢弃）：

$$\begin{array}{r}
 0 \\
 1011 + 1 \\
 0110 = \underbrace{1}_{\text{进位丢弃}} 0 \\
 0001
 \end{array} \quad (4.1)$$

丢掉进位后，结果为 0001，即 +1。验证： $(+11) + (-10) = +1$ ，完全正确！

补码的精妙之处在于：符号位参与运算，进位自然丢弃，加减法统一为加法，大大简化了 CPU 的运算电路设计。现代计算机内部整数运算几乎都采用补码。

4.1.3 原码乘法的易错点

例 用原码直接做乘法 $(+2) \times (-3) = -6$

5 位原码：

0

+2 的原码：0010

1

-3 的原码：0011

逐位相乘（不考虑符号位）： $0010 \times 0011 = 00110$ （即 6）

符号位处理：同号得正，异号得负。 $0 + 1 = 1$ （负）

1

最终结果：0110，即 -6。验证： $(+2) \times (-3) = -6$ ，正确。

但注意：原码乘法中符号位不能直接参与乘法运算，必须单独处理。这正是补码运算的优势所在：补码可以统一处理符号，无需特殊逻辑。

4.2 存储器层次结构

直觉 你不妨想：如果你的书桌能放 10 本书，书架能放 100 本，图书馆能放 10000 本。书桌上的书随手可取（快），但容量小；图书馆的书很多（大），但取一本要走很远（慢）。计算机的存储器也是这样的金字塔结构。

从快到慢、从小到大，存储器层次如下：

层次	速度	容量	角色
寄存器 (Register)	最快	最小	CPU 内部，直接参与运算
Cache (缓存)	很快	小	寄存器与内存之间的缓冲
内存 (Memory)	快	中	程序运行时的主存储
磁盘 (Disk)	慢	大	持久化存储
磁带 (Tape)	最慢	最大	归档备份

核心规律：容量越来越大，每位价格越来越便宜，但访问速度越来越慢。这就是存储器层次结构的设计哲学：用少量快速存储器缓存热点数据，用大量慢速存储器保存冷数据，在速度和成本之间取得平衡。

4.2.1 存储单位换算

例 存储单位从字节开始，每次乘以 $2^{10} \approx 10^3$ ：

单位	名称	实际字节数	近似值
B	字节 (Byte)	1	1
KB	千字节	2^{10}	$\approx 10^3$
MB	兆字节	2^{20}	$\approx 10^6$
GB	千兆字节	2^{30}	$\approx 10^9$
TB	兆兆字节	2^{40}	$\approx 10^{12}$

取小数值算给你看：1 KB = 2^{10} = 1024 字节，而 10^3 = 1000，两者差 24 字节。虽然近似，但在性能计算时务必区分二进制幂与十进制幂。例如硬盘厂商通常用十进制 (1 TB = 10^{12} 字节)，而操作系统用二进制 (1 TiB = 2^{40} 字节)，这就是为什么你买的“1TB 硬盘”在系统里显示不到 1TB 的原因。

4.3 计算机系统分层与指令周期

计算机系统分为两大子系统：

1. **硬件系统**：处理器系统（主机）、存储器系统、外部设备（输入/输出设备）
2. **软件系统**：系统软件（操作系统、编程语言、工具软件）、应用软件（办公软件等）

CPU 执行一条指令分为三个步骤，称为**指令周期** (Instruction Cycle)：

1. **取指令** (Fetch)：从内存中读取下一条要执行的指令
2. **分析指令** (Decode)：解读指令的操作码和操作数
3. **执行指令** (Execute)：完成指令指定的操作

直觉 不妨把这个过程想象成做菜：先从食谱上读下一步做什么（取指令），理解这步要什么材料什么操作（分析指令），然后实际动手做（执行指令）。CPU 就这样一条一条地执行指令，周而复始。理解指令周期是分析处理器性能瓶颈的基础：如果取指令太慢（内存瓶颈），或者分析指令太慢（分支预测失败），整个流水线就会停滞。

5 网络基础

超算集群由成百上千个节点组成，节点之间需要高速网络通信。理解网络基础是配置和管理集群的前提。

5.1 TCP/IP 协议

直觉 你寄一个包裹给远方的朋友，需要做两件事：一是把物品打包、贴上收件人地址（这是 IP 的工作），二是选择快递公司、挂号保价、丢失了要重新寄（这是 TCP 的工作）。TCP 和 IP 分工合作，共同完成网络数据传输。

TCP/IP 由 Vint Cerf（文特·瑟夫）和 Bob Kahn（鲍勃·卡恩）设计，源自 1969 年美国国防部主导的 ARPANET，是事实上（de facto）的互联网标准。它采用 4 层结构，比 OSI 七层模型简单。

- 1. **TCP**（传输控制协议，Transmission Control Protocol）：负责数据打包、编号，在接收端将数据按原格式组装。TCP 还负责错误检测和重传，保证数据可靠到达。如果某个数据包出错或丢失，TCP 会要求重新发送。
- 2. **IP**（网络互连协议，Internet Protocol）：为每个数据包加上接收机地址后在网络信道中传输。IP 只管把数据包送到目的地，不管数据是否完整或有序。

初学者常把 TCP 和 IP 混为一谈。记住：TCP 管端到端可靠性（数据对不对），IP 管网络层传输（数据到不到）。两者职责分离，各司其职。TCP/IP 是一个协议集，包含数以百计的各种协议的集合。

5.2 IP 地址与子网掩码

5.2.1 IP 地址

IP 协议规定：每台入网的计算机都必须有一个唯一的网络地址，即 **IP 地址**（IP Address）。IP 地址是一个 4 字节（32 位）的地址，通常用点分十进制表示。

例 IP 地址 210.32.133.150 的二进制表示：

11010010

10011011

10000101

00100000

字节 1

字节 2

字节 3

字节 4

(5.1)

点分十进制：210.32.133.150

拥有 IP 地址的机器叫**主机**（Host），路由器根据 IP 地址传输数据包。

5.2.2 专用地址范围

有些 IP 地址只能用于一个机构的内部通信，不能用于和互联网上的主机通信，这些叫**专用地址**（Private Address）。路由器对目的地址是专用地址的数据报一律不转发。

范围	CIDR	地址区间
A 类专用	10.0.0.0/8	10.0.0.0 10.255.255.255
B 类专用	172.16.0.0/12	172.16.0.0 172.31.255.255
C 类专用	192.168.0.0/16	192.168.0.0 192.168.255.255

配置集群网络时务必正确区分内网与公网地址。集群节点间通信通常使用专用地址，对外服务才使用公网地址。

5.2.3 子网与子网掩码

把一个较大的网络分成若干个较小的网络，通过路由器连接起来，这些具有相同网络标识的小网络称为**子网**（Subnet）。划分子网后，要告诉网络子网是如何划分的，这就是**子网掩码**（Subnet Mask）。

原来 IP 地址的“网络-主机”结构变成了“网络-子网-主机”结构。子网掩码的作用是调整网络地址和主机地址的平衡。

地址类别	默认子网掩码
A 类	255.0.0.0
B 类	255.255.0.0
C 类	255.255.255.0

子网掩码的核心用途是：判断目标主机是否在同一子网内。方法是將本机 IP 与目标 IP 分别与子网掩码做逻辑“与”操作，若结果相同则目标在网内可直接通信，否则需经路由器转发。

例 判断 192.168.1.100 和 192.168.1.200 是否在同一子网（掩码 255.255.255.0）：

192.168.1.100 AND 255.255.255.0 = 192.168.1.0

192.168.1.200 AND 255.255.255.0 = 192.168.1.0

两者结果相同（都是 192.168.1.0），说明在同一子网内，可以直接通信。

如果把掩码换成 255.255.255.128，则：

192.168.1.100 AND 255.255.255.128 = 192.168.1.0

192.168.1.200 AND 255.255.255.128 = 192.168.1.128

结果不同（192.168.1.0 vs 192.168.1.128），说明不在同一子网，需要路由器转发。

这是集群节点间通信诊断的基础技能。当你发现两个节点无法直接通信时，第一步就是检查它们的 IP 和子网掩码是否在同一网段。

5.3 CIDR：无类别域间路由

传统的 A/B/C 类地址采用固定掩码长度，灵活性差。**CIDR**（无类别域间路由，Classless Inter-Domain Routing）打破了传统固定掩码限制，允许灵活的地址划分。

CIDR 使用 /n 后缀表示网络前缀长度，例如 192.168.1.0/24 表示前 24 位是网络号，后 8 位是主机号，相当于传统的 C 类掩码 255.255.255.0。而 172.16.0.0/12 表示前 12 位是网络号，后 20 位是主机号。

CIDR 的好处是可以根据实际需要划分任意大小的子网，避免 IP 地址浪费。例如一个需要 500 个地址的网络，可以用 /23 ($2^9 = 512$ 个地址)，而不需要浪费一整个 B 类网络 (65536 个地址)。

6 操作系统核心功能

操作系统（Operating System, OS）管理硬件资源，为用户程序提供统一接口。它处于用户接口与用户/程序之间，核心是**内核**（Kernel），是软硬件之间的桥梁。

直觉 操作系统就像一个大楼管理员。大楼里有 CPU（会议室）、内存（办公位）、磁盘（仓库）、各种设备（电梯、空调）。管理员负责：谁什么时候用会议室（CPU 调度），谁坐哪个办公位（内存管理），东西存在仓库哪里（文件系统），怎么操作电梯和空调（设备驱动）。住户（用户程序）只需要告诉管理员需求，不需要自己去管理这些资源。

操作系统的核心功能包括：

1. **CPU 调度**：决定哪个进程何时使用 CPU，保证公平和效率
2. **内存管理**：分配和回收内存空间，包括虚拟内存、分页等机制
3. **文件系统**：组织和管理磁盘上的数据，提供文件和目录的抽象
4. **设备驱动**：与硬件设备交互，屏蔽硬件细节

例 多线程的“搬家”直觉

操作系统中的多线程（Multithreading）可以用搬家来理解：一个人搬家（单线程）要一件一件搬，很慢；叫上几个朋友一起搬（多线程），每人搬不同的东西，就快多了。但朋友之间要协调，不能两个人同时去搬同一件东西（数据竞争），也不能把东西放错位置（同步问题）。这正是后续 OpenMP 并行编程要解决的核心问题。

7 要点速查

要点	说明
HPC	超级计算，利用大量处理器协同完成大规模计算
云计算	算力按需供应，底层依赖超算能力，“第五大基础设施”
三大竞赛	ASC（中国，规模最大）/ ISC（德国）/ SC（美国），获奖率 1t 5%
HPL	稠密线性方程组基准测试，Top500 排名标准
e Prize	指定应用大规模并行优化，“青年人才的戈登·贝尔奖”
并行计算	任务拆分到多处理单元同时执行，缩短运行时间
并行三栈	OpenMP（多线程）/ MPI（多进程）/ CUDA（GPU）
补码	减法转加法，符号位参与运算，进位丢弃
存储层次	寄存器 → Cache → 内存 → 磁盘 → 磁带，速度递减容量递增
存储单位	$1 \text{ KB} = 2^{10} \approx 10^3$ ，区分二进制幂与十进制幂
指令周期	取指令 → 分析指令 → 执行指令
TCP vs IP	TCP 管可靠性（数据对不对），IP 管路由（数据到不到）
专用地址	$10.x / 172.16-31.x / 192.168.x$ ，路由器不转发
子网掩码	与本机和目标 IP 做逻辑与，结果相同则同网内
CIDR	无类别域间路由，/n 表示网络前缀长度，打破 A/B/C 类限制
OS 内核	CPU 调度 / 内存管理 / 文件系统 / 设备驱动

8 本章你将学会

1. 能够解释什么是超级计算，以及云计算与超算的关系
2. 能够说出世界三大超算竞赛的名称和特点，以及 HPL 与 Top500 榜单的含义

3. 能够用补码完成正负整数的加减运算，并理解补码统一加减法的优势
4. 能够判断两个 IP 地址是否在同一子网内，并解释 CIDR 的作用
5. 能够描述存储器层次结构和指令周期的三个阶段，理解它们对性能优化的意义

9 小结

本章我们从“为什么需要超算”出发，介绍了超级计算的基本概念：从云计算的“自来水”模型，到超算的应用领域和国家战略意义；从世界三大超算竞赛和浙大超算队的历程，到 HPL 基准测试与 Top500 榜单；从并行计算的核心思想（矩阵加法 vs 向量点积），到三大并行技术栈（OpenMP/MPI/CUDA）的定位。

在计算机基础部分，我们回顾了补码运算（减法变加法）、存储器层次结构（金字塔模型）、存储单位换算（ $2^{10} \approx 10^3$ ）、指令周期（取指-分析-执行），以及网络基础（TCP/IP 分工、IP 地址与子网掩码、CIDR）和操作系统核心功能。

作为 HPC 新手，最重要的两件事是：第一，建立“并行”的思维方式，把大任务拆成小任务同时干；第二，理解计算机系统的层次结构，因为性能优化的本质就是在存储层次和指令周期上做文章。后续章节将深入 OpenMP、MPI、CUDA 等具体的并行编程技术，让我们从“理解超算”走向“驾驭超算”。

讲义基于 HPC101-2025 Day1「超算概述」课程内容编写